



8

31

4650 sp

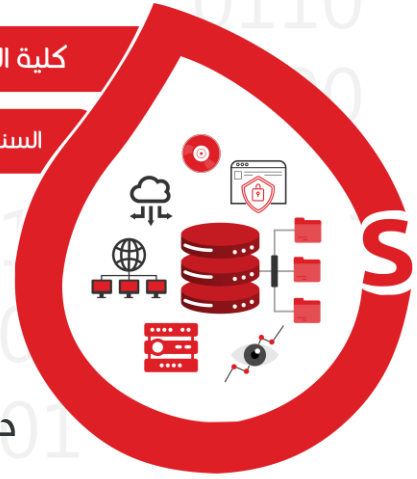
نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Transaction

د. مادلين عبود



RB Informatcs; 23/11/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

أفكار المحاضرة:

- مفهوم المناقلة transaction concept
- حالة مناقلة transaction state
- الكتلية والاستمرارية atomicity and durability
- التنفيذ بشكل قابل للتسلسل serializability
- قابلية الاسترجاع والاستعادة recoverability
- العزل implementation of isolation
- تعريف المناقلات في SQL - sql transaction definition in
- اختبار إمكانية تسلسل التنفيذ testing for serializability

تذكرة أريسيزية:

خصائص ACID:

- الذرية Atomicity: تنفيذ المعاملة كوحدة واحدة، إما بالكامل أو لا شيء
- الاتساق Consistency (وردت في المحاضرة باسم الملائمة): الحفاظ على قاعدة بيانات صحيحة ومتسقة قبل وبعد التنفيذ، مثل شرط عدم تجاوز الرصيد حدًا معينًا
- العزل Isolation: منع تداخل نتائج المعاملات المتزامنة بحيث تبدو وكأنها نُفذت بشكل متسلسل
- الاستمرارية Durability: بقاء التغييرات بعد عمل commit حتى في حال حدوث أعطال

مفهوم المناقلة transaction concept

المعاملة (Transaction) هي وحدة تنفيذ برمجية قد تشمل أكثر من تعليمة، وتقوم بالوصول إلى عناصر بيانات مختلفة وربما تعديلها. أثناء التنفيذ يمكن أن تمر قاعدة البيانات بحالة وسيطة غير متسقة، لكن هذه الحالة ليست نهائية؛ فإذا حدث خطأ أو عطل في النظام أو الأجهزة يمكن الرجوع إلى الحالة السابقة عبر الـ Rollback، أما إذا اكتملت المعاملة بنجاح فيتم تثبيت التغييرات بشكل دائم عبر الـ Commit. ولضمان سلامة البيانات يجب أن تحقق المعاملة خصائص ACID.

إضافة إلى ذلك، هناك نقطتان أساسيتان في التعامل مع المعاملات:

1. التعامل مع الأعطال:

حيث يجب أن يكون النظام قادراً على الاسترجاع أو إعادة التنفيذ بشكل كامل، وليس بشكل جزئي، عند حدوث فشل.

2. التحكم في التزامن (Concurrency Control):

عند تنفيذ عدة معاملات بالتوازي، يجب ضمان أن النتائج النهائية تعادل النتائج كما لو أن المعاملات نُفذت بشكل متسلسل، وذلك لتجنب التضارب وضمان الاتساق.

لماذا نحتاج التحكم بالوصول المتزامن لعدة مناقلات معاً؟

نحتاج إلى التحكم بالوصول المتزامن لأن تنفيذ عدة معاملات في نفس الوقت قد يؤدي إلى أخطاء مثل القراءات الخاطئة، فقدان التحديثات، أو فقدان الاتساق Concurrency Control. يضمن أن قاعدة البيانات تبقى صحيحة وأمنة حتى مع وجود معاملات متعددة وأعطال محتملة.

مثال 1:

- T1: تقرأ قيمة A، تطرح منها N، تكتب القيمة الجديدة، ثم تقرأ B وتضيف إليه N أي أنها تمثل عملية تحويل مبلغ N من الحساب A إلى الحساب B.
- T2: تقرأ قيمة A، تضيف إليها M، ثم تكتب القيمة الجديدة. أي أنها تمثل عملية إيداع مبلغ M في الحساب A.

المشكلة عند التنفيذ المتزامن:

- إذا نُفذت المعاملتان بشكل متداخل (بدون ضبط للترتيب)، قد يحدث أن:
 - T1 تقرأ قيمة A وتطرح N
 - قبل أن تكتب T1 القيمة الجديدة، تأتي T2 وتقرأ نفس القيمة القديمة لـ A وتضيف إليها M
 - ثم تكتب T2 النتيجة، وبعدها تكتب T1 نتیجتها أيضاً
- النتيجة النهائية تصبح A-N أو A+M حسب ترتيب الكتابة:

T1	T2
Read (A) A := A - N	
	Read (A) A := A + M
Write (A) Read (B)	
	Write (A)
B := B + N Write (B)	

القيمة المخزنة في A غير صحيحة ←

مثال 2:

- T1: تقرأ A، تطرح منه N، تكتب القيمة الجديدة
- T2: تقرأ A، تضيف إليه M، ثم تكتب القيمة الجديدة

المشكلة:

- عند التنفيذ المتزامن، قد يحدث أن:

- T1 تقرأ A وتطرح N
- T2 هدفها هو ان تقرأ نفس القيمة القديمة ل A وتضيف إليها M.
- بسبب التنفيذ المتزامن قرأت T2 نتيجة تعديل T1 ل A

- النتيجة النهائية تصبح غير صحيحة، لأن القيمة النهائية ل A ليست $A-N+M$ كما يجب أن تكون، بل إما $A-N$ ($t1$) أو $A+M$ ($t2$) حسب ترتيب الكتابة
- هذا يؤدي إلى قراءة خاطئة (Incorrect Read) من قبل T2، وبالتالي قاعدة البيانات تصبح غير متسقة (غير متلائمة inconsistent).

T1	T2
Read (A) $A := A - N$ Write (A)	
	Read (A) $A := A + M$ Write (A)
Read (A) Transaction fails must change value of A to old value	

← T2 قامت بقراءة قيمة وسيطة خاطئة ←

لماذا نحتاج إلى الاسترجاع أو الاستعادة (recovery)؟

- عند حدوث عطل في النظام أو خطأ أثناء المعاملة أو حتى مشكلات في القرص أو الأجهزة، قد تتوقف المعاملة في منتصف التنفيذ.
- إذا لم يكن هناك آلية للاسترجاع، ستبقى قاعدة البيانات في حالة غير متسقة، لأن بعض التغييرات الجزئية قد تكون نفذت بينما أخرى لم تنفذ.
- الاسترجاع يضمن أن قاعدة البيانات إما تعود إلى الحالة السابقة (Rollback) أو تكمل بشكل صحيح (Commit)، فلا تبقى في حالة وسيطة خاطئة.

أنواع الأعطال التي تستدعي الاسترجاع:

1. System crash انهيار النظام أو توقفه المفاجئ.
2. Transaction error خطأ أثناء تنفيذ المعاملة نفسها/ أخطاء محلية أو شروط استثنائية مكتشفة من قبل المناقلة
3. Disk failure أعطال في القرص الصلب أو فقدان البيانات
4. Physical problems مشاكل مادية في الأجهزة

دور مدير الاسترجاع (Recovery Manager):

- مسؤول عن متابعة المعاملات وضمان تنفيذها بشكل صحيح.
- يقوم بالخطوات التالية:
 1. Begin transaction: بدء المعاملة.
 2. Read / Write: تنفيذ عمليات القراءة والكتابة
 3. End transaction: إنهاء المعاملة.
 4. Commit: تثبيت التغييرات بشكل دائم.
 5. Rollback (Abort): إلغاء التغييرات وإرجاع البيانات إلى حالتها السابقة عند الفشل.

مثال لتوضيح خصائص ACID التي تضمن سلامة المعاملات في قواعد البيانات:

ليكن لدينا معاملة ل تحويل 50 دولار من الحساب A إلى الحساب B:

خطوات المعاملة:

- | | |
|------------------------------|------------------------------|
| 1. قراءة قيمة الحساب A | 4. قراءة قيمة الحساب B. |
| 2. طرح 50 من A | 5. إضافة 50 ل B |
| 3. كتابة القيمة الجديدة لـ A | 6. كتابة القيمة الجديدة لـ B |

خصائص ACID في المثال:

Atomicity الذرية:

إذا فشلت المعاملة بعد خصم 50 من A وقبل إضافة 50 إلى B، يجب ألا تعتمد التغييرات الجزئية. النظام يعيد الحالة السابقة (Rollback) لتجنب أن تصبح القاعدة غير متلائمة.

Consistency الاتساق:

مجموع الحسابين (A + B) يجب أن يبقى ثابتاً قبل وبعد المعاملة. التحويل لا يخلق أو يضيع المال، بل ينقله فقط.

Isolation العزل:

إذا سمح النظام لمعاملة أخرى بالوصول إلى قاعدة البيانات بين الخطوة 3 والخطوة 6، ستقرأ قاعدة بيانات غير متسقة (المجموع سيكون أقل من المطلوب). الحل البسيط هو التنفيذ التسلسلي، لكن التنفيذ المتوازي له فوائد كبيرة إذا تم التحكم فيه بشكل صحيح.

Durability الاستمرارية:

بعد إتمام المعاملة وإبلاغ المستخدم بنجاح التحويل، يجب أن تبقى التغييرات محفوظة حتى لو حدثت أعطال في النظام. أي أن التحديثات تصبح دائمة ولا تُفقد.

Transaction state حالة مناقلة

حالات المعاملة:

1. Active نشطة:

- الحالة الابتدائية.
- المعاملة تكون في طور التنفيذ وتنفذ التعليمات خطوة بخطوة.

2. Partially Committed جزئياً مكتملة:

- تحدث بعد تنفيذ آخر تعليمة في المعاملة، لكن قبل التأكد من تثبيت النتائج بشكل دائم.
- قاعدة البيانات قد تكون جاهزة للانتقال إلى الالتزام (Commit).

3. Failed فشلت:

- تحدث عند اكتشاف أن التنفيذ الطبيعي لا يمكن أن يستمر (بسبب خطأ منطقي أو عطل في النظام).
- المعاملة لا تستطيع إكمال خطواتها.

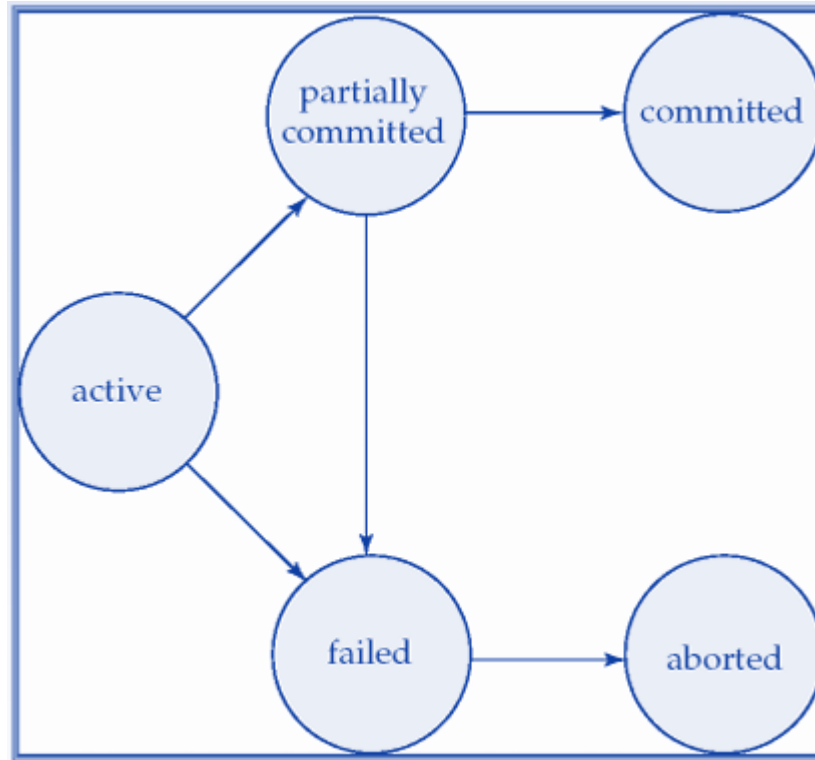
4. Aborted ملغاة:

- تحدث بعد إرجاع قاعدة البيانات إلى حالتها السابقة باستخدام Rollback.
- هناك خياران بعدها:
 - إعادة تشغيل المعاملة إذا لم يكن هناك خطأ داخلي منطقي.
 - إنهاء المعاملة نهائياً (Kill).

5. Committed مكتملة:

- تحدث بعد نجاح المعاملة بالكامل.
- التغييرات تصبح دائمة في قاعدة البيانات حتى لو حدثت أعطال لاحقاً.

القرار الذي تتخذه اليوم، قد يغير القصة كلها غداً

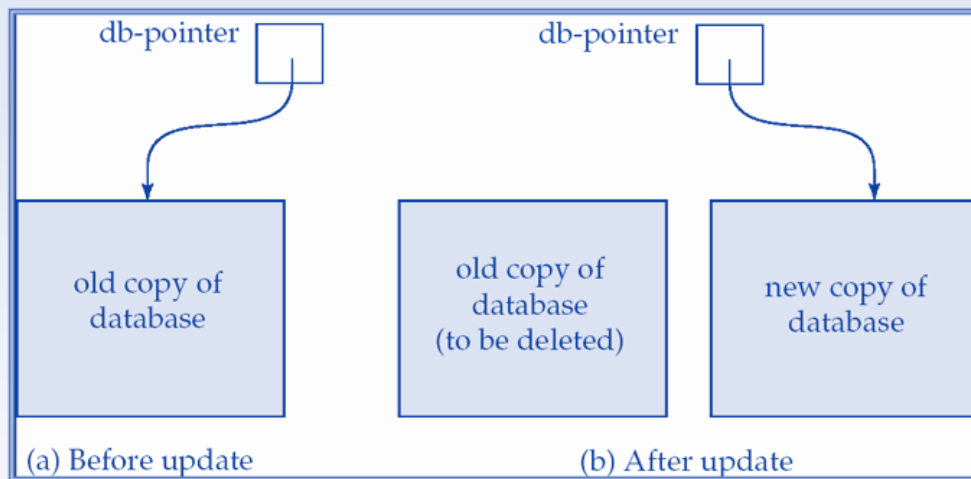


شرح الشكل: المعاملة تبدأ نشطة (Active)، ثم تصبح جزئياً مكتملة (Partially Committed) بعد تنفيذ آخر تعليمة. إذا نجحت، تنتقل إلى حالة Committed وثبتت التغييرات. أما إذا فشلت، تنتقل إلى حالة Failed ثم Aborted حيث تلغى التغييرات ويُعاد النظام إلى حالته السابقة.

الكتلية والاستمرارية atomicity and durability

يتم تحقيق خاصيتي الكتلية/الذرية (Atomicity) والاستمرارية (Durability) في قواعد البيانات باستخدام تقنية تسمى (Shadow paging) ضمن آليات إدارة الاسترجاع (Recovery management).

The shadow-database scheme:



الفكرة الأساسية:

- الهدف هو ضمان أن المعاملة (Transaction) إما تُنفذ بالكامل أو لا تُنفذ إطلاقاً (Atomicity)، وأن نتائجها تبقى محفوظة حتى في حال حدوث أعطال (Durability).
- أسلوب قاعدة البيانات الظلية (Shadow Database) يقوم على وجود نسختين من قاعدة البيانات:
 1. نسخة فعالة (Active) تُستخدم أثناء تنفيذ المعاملة
 2. نسخة متسقة (Consistent) يُشير إليها مؤشر (db_pointer) وتعتبر النسخة الرسمية المعتمدة.

كيفية العمل:

- عند بدء معاملة جديدة، يتم إنشاء نسخة ظل (Shadow Copy) من قاعدة البيانات.
- التعديلات تُجرى على النسخة الفعالة فقط.
- إذا اكتملت المعاملة بنجاح، يتم تحديث المؤشر (db_pointer) ليُشير إلى النسخة الجديدة، فتصبح هي النسخة الرسمية.
- إذا حدث فشل أو عطل، يبقى المؤشر على النسخة القديمة المتسقة، وبالتالي تُلغى التغييرات غير المكتملة.

المزايا والقيود:

- المزايا:
 - طريقة بسيطة ومباشرة لضمان الذرية والاستمرارية.
 - لا تحتاج إلى سجلات معقدة أو عمليات استرجاع طويلة.
- القيود:
 - تفترض عدم وجود أعطال في القرص أثناء العملية.
 - غير عملية في قواعد البيانات الضخمة لأنها تتطلب نسخاً كاملة من البيانات، مما يجعلها مكلفة من حيث الأداء والمساحة.
 - تستخدم غالباً في الحالات البسيطة أو قواعد البيانات الصغيرة.

فوائد تنفيذ المعاملات بشكل متزامن

1. زيادة الإنتاجية (Throughput):

تنفيذ أكثر من معاملة في نفس الوقت يرفع عدد العمليات التي يمكن للنظام معالجتها.

2. تقليل زمن الاستجابة (Response Time):

متوسط الزمن اللازم لإنهاء المعاملات يصبح أقل مقارنة بالتنفيذ التسلسلي البحت.

3. استغلال أفضل الموارد:

مثل المعالج والقرص، حيث لا تبقى الأجهزة في حالة انتظار طويلة.

إن لم تقاتل لأجل حلمك، ستتعب في خدمة حلم غيرك

تقنيات تحقيق العزل

- لضمان أن التنفيذ المتزامن لا يؤدي إلى أخطاء أو فقدان الاتساق، تُستخدم تقنيات مثل:
 - مخططات التحكم في التزامن (Concurrency Control Schedules)
 - هذه المخططات تحدد ترتيب التعليمات بحيث تكون النتيجة النهائية مكافئة للتنفيذ التسلسلي

تعريف مخططات التنفيذ (Schedules)

- هي تسلسل يبين الترتيب المنطقي للتعليمات من مجموعة معاملات تُنفذ معاً.
- يجب أن تظهر مخططات التنفيذ جميع التعليمات لكل معاملة، وتحافظ على ترتيب التعليمات داخل كل معاملة.
- الهدف هو ضمان أن قاعدة البيانات تبقى متسقة بعد التنفيذ.

مثال 1:

- T1: تحويل 50 دولار من الحساب A إلى الحساب B.
- T2: تحويل 10٪ من رصيد الحساب A إلى الحساب B.

تظهر عدة أشكال لمخططات التنفيذ هنا:

1. Schedule مخطط تنفيذ تسلسلي T1 ثم T2 <T1, T2>:

- تُنفذ المعاملة T1 بالكامل أولاً، ثم تُنفذ T2
- النتيجة النهائية صحيحة، حيث يبقى مجموع الحسابين A+B ثابتاً.

■ A serial schedule in which T_1 is followed by T_2 :

T_1	T_2
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B)

2. Schedule مخطط تنفيذ تسلسلي T_2 ثم T_1 < T_2, T_1 :

- تنفذ المعاملة T_2 بالكامل أولاً، ثم تنفذ T_1
- أيضاً يحافظ على الاتساق، والمجموع $A+B$ يبقى ثابتاً.

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$

3. Schedule مخطط تنفيذ غير تسلسلي لكنه مكافئ ل Schedule 1:

- هنا يتم التداخل بين تعليمات T_1 و T_2 ، لكن بطريقة تجعل النتيجة النهائية مساوية كما لو نُفذت T_1 ثم T_2 بشكل تسلسلي.
- مخطط التنفيذ هذا يُعتبر **Serializable** لأنه مكافئ لجدول تسلسلي.
- المجموع $A+B$ يبقى محفوظاً.

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1.

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$
$\text{read}(B)$ $B := B + 50$ $\text{write}(B)$	$\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$

4. Schedule مخطط تنفيذ متزامن غير صحيح:

- في هذا المخطط، يحدث تداخل غير مضبوط بين تعليمات T_1 و T_2 .
- النتيجة النهائية لا تحافظ على قيمة $A+B$ ، أي أن قاعدة البيانات تصبح غير متسقة.
- هذا مثال على مخطط تنفيذ غير قابل للتسلسل (Non-serializable).

- The following concurrent schedule does not preserve the value of $(A + B)$.

T_1	T_2
read(A) $A := A - 50$	
	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B)
write(A) read(B) $B := B + 50$ write(B)	
	$B := B + temp$ write(B)

نستنتج أن: يجب أن يكون مخطط التنفيذ مكافئ لمخطط التنفيذ التسلسلي (Serializability).

التنفيذ بشكل قابل للتسلسل serializability

الفرضية الأساسية التي سنبنى عليها:

- كل معاملة منفردة تحافظ على اتساق قاعدة البيانات.
- إذا نُفذت مجموعة معاملات بشكل تسلسلي (واحدة بعد الأخرى)، فإن قاعدة البيانات تبقى متسقة.

تعريف التسلسلية (Serializability):

- ذكرنا أن مخطط التنفيذ (Schedule) هو ترتيب تنفيذ التعليمات (قراءة/كتابة) من عدة معاملات.
- مخطط التنفيذ يُعتبر قابل للتسلسل Serializable إذا كانت نتيجته مكافئة لمخطط تسلسلي (متعاقب).
- أي أن التنفيذ المتزامن يجب أن يعطي نفس النتيجة كما لو نُفذت المعاملات بشكل متسلسل.

أنواع التسلسل:

1. Conflict Serializability التسلسل بالتصادم:

- يعتمد على تحليل التعارضات بين التعليمات (قراءة/كتابة) على نفس العنصر.

2. View Serializability التسلسل ظاهرياً:

يعتمد على القيم المقروءة والمكتوبة.

لتبسيط النموذج في أثناء الشرح:

- يتم تجاهل العمليات الأخرى غير القراءة والكتابة.
- يُفترض أن المعاملات قد تقوم بحسابات محلية بين عمليات القراءة والكتابة، لكن هذه الحسابات لا تؤثر على الاتساق العام.
- لذلك، الجداول المبسطة تتكون فقط من تعليمات Read و Write.

متى تكون التعليمات متعارضة/متصادمة:

- يحدث التعارض بين تعليمتي T_i و T_j إذا كانتا تصلان إلى نفس العنصر Q ، وكان على الأقل إحداها عملية كتابة.
- الحالات:

1. $read(Q), read(Q)$: لا يوجد تعارض
2. $read(Q), write(Q)$: يوجد تعارض
3. $write(Q), read(Q)$: يوجد تعارض
4. $write(Q), write(Q)$: يوجد تعارض

الفكرة: وجود تعارض يفرض ترتيباً زمنياً منطقياً بين التعليمات، بحيث لا يمكن تبديلها دون تغيير النتيجة. أما إذا لم يكن هناك تعارض، يمكن تبديل التعليمات دون أن تتأثر النتيجة.

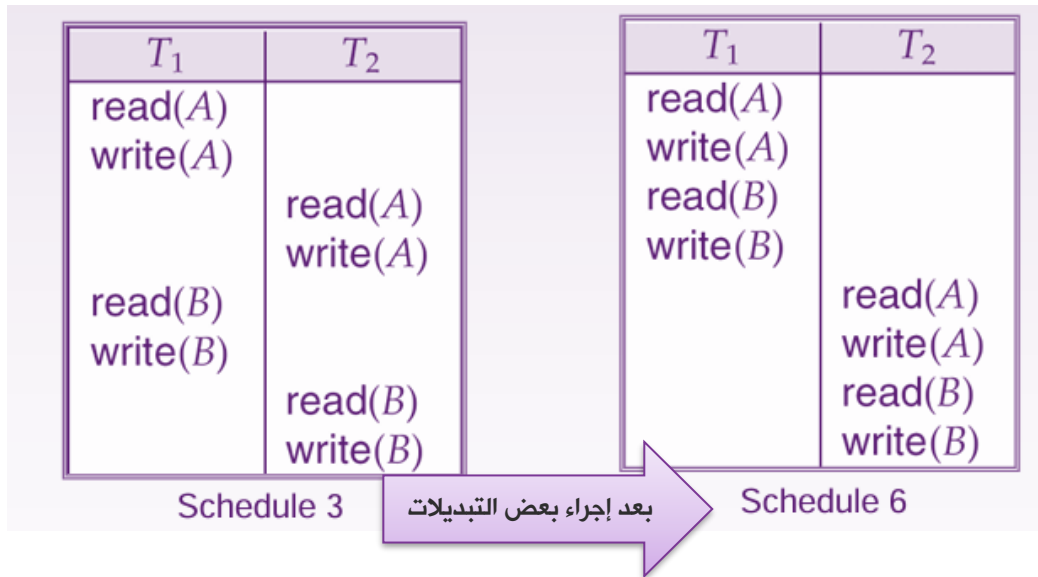
التسلسل بالتصادم (Conflict Serializability)

- إذا كان لدينا مخطط تنفيذ S يمكن تحويله إلى مخطط تنفيذ آخر S' عن طريق سلسلة من عمليات تبديل التعليمات غير المتعارضة، نقول إن الجدولين متكافئان بالتصادم (Conflict Equivalent).
- بمعنى آخر:** نقول إن s و s' مخطوطا تنفيذ متكافئين تصادمياً Conflict Equivalent إذا نتج أحدهم عن الآخر بعد سلسلة من التبديلات بين التعليمات غير المتصادمة.
- إذا كان مخطط التنفيذ S مكافئاً تصادمياً Conflict Equivalent لمخطط تنفيذ تسلسلي (متعاقب)، نقول إنه مخطط متسلسل تصادمياً (Conflict Serializable).

الطريق الصعب لا يَنْهَكَ، بل يصنع منك إنساناً لا يُهْزَم

أمثلة:

- Schedule 3 → Schedule 6 : يمكن تحويل مخطط التنفيذ 3 إلى مخطط تنفيذ تسلسلي (مخطط التنفيذ 6 حيث T2 بعد T1) عبر تبديل التعليمات غير المتعارضة.



لذلك نقول أن مخطط التنفيذ 3 متسلسل تصادياً (Conflict Serializable).

- مخطط تنفيذ غير قابل للتسلسل بالتصادم: في بعض الجداول، لا يمكن تبديل التعليمات للوصول إلى جدول تسلسلي مثل $\langle T_3, T_4 \rangle$ أو $\langle T_4, T_3 \rangle$ في هذه الحالة، مخطط التنفيذ غير قابل للتسلسل بالتصادم (غير متسلسل تصادياً)، وبالتالي قد يؤدي إلى نتائج غير صحيحة أو فقدان الاتساق:

T ₃	T ₄
read(Q)	
write(Q)	
	write(Q)

توضيح للمثال:

- مخطط التنفيذ التسلسلي هو الذي تنفذ فيه المعاملات واحدة بعد الأخرى بشكل كامل، مثل:
 - $\langle T_3, T_4 \rangle$ أي تنفيذ T3 ثم T4.
 - $\langle T_4, T_3 \rangle$ أي تنفيذ T4 ثم T3.

المشكلة:

في هذا المثال، لا يمكننا تبديل التعليمات في الجدول للوصول إلى أي من الجدولين التسلسليين السابقين.

السبب هو وجود تعليمات متعارضة (Conflicting Instructions) بين T3 و T4 تجعل من المستحيل إعادة ترتيب الجدول دون تغيير النتائج.

النتيجة: مخطط التنفيذ غير متسلسل تصادياً.

التسلسل ظاهرياً View Serializability

هو امتداد لمفهوم التسلسل بالتصادم (Conflict Serializability) لكنه أوسع منه.

الفكرة الأساسية:

- لدينا مخططان للتنفيذ S و S' يحتويان على نفس مجموعة المعاملات.
 - نقول إنهما متكافئان ظاهرياً (View Equivalent) إذا تحققت ثلاثة شروط:
 1. إذا قرأت معاملة قيمة ابتدائية لعنصر Q في مخطط التنفيذ S ، يجب أن تقرأ نفس القيمة الابتدائية في مخطط التنفيذ S'
 2. إذا قرأت معاملة قيمة ناتجة من معاملة أخرى في مخطط التنفيذ S ، يجب أن تقرأ نفس القيمة الناتجة من نفس المعاملة في مخطط التنفيذ S'
 3. إذا كانت هناك معاملة تنفذ آخر عملية كتابة على العنصر Q في مخطط التنفيذ S ، يجب أن تكون نفس المعاملة هي التي تنفذ آخر عملية كتابة على Q في مخطط التنفيذ S'
- إذاً، التكافؤ ظاهرياً يعتمد فقط على عمليات القراءة والكتابة، وليس على ترتيب التعليمات أو التعارضات الزمنية.

- الجدول يكون قابلاً للتسلسل بالرؤية / متسلسل ظاهرياً View Serializable إذا كان مكافئاً ظاهرياً (View Equivalent) لجدول تسلسلي.

مثال على مخطط متسلسل ظاهرياً ولكن غير متسلسل تصادميةً:

T_3	T_4	T_6
read(Q)	write(Q)	write(Q)
write(Q)		

ملاحظة:

كل مخطط تنفيذ متسلسل تصادميةً هو أيضاً متسلسل ظاهرياً و لكن العكس غير صحيح.

Every conflict serializable schedule is also view serializable

الحياة لا تعطي الأفضل، تعطي من يصبر أكثر

باختصار:

في التسلسل التصادمي (Conflict Serializability):

← نهتم بكل التعارضات (read/write) و (write/write)

لكن في التسلسل الظاهري (View Serializability):

← نهتم فقط بـ:

1. من قرأ القيمة الابتدائية initial read

2. من قرأ من من read-after-write

3. من قام بآخر كتابة final write

فائدة اربيسيزية

1. Initial Read القراءة الابتدائية:

هي أول عملية قراءة لعنصر بيانات معيّن في الجدول.

الفكرة المهمة:

- عنصر البيانات قد يُقرأ عدة مرات أثناء التنفيذ
- لكن أول قراءة فقط هي التي تُسمّى Initial Read
- هذه القراءة تعني أن المعاملة قرأت القيمة الأصلية من قاعدة البيانات

الربط مع شرط التكافؤ ظاهرياً:

إذا كانت معاملة معينة مثل T1 هي أول من قرأ العنصر Z في جدول Y1، فلا بد أن تكون نفس المعاملة T1 هي أول من يقرأ Z في الجدول Y2.

السبب: حتى تبدأ المعاملات في الجدولين من نفس نقطة البيانات.

2. Final Write الكتابة النهائية:

هي آخر عملية write على عنصر البيانات في الجدول.

الربط مع شرط التكافؤ ظاهرياً:

إذا كانت T2 هي آخر معاملة تكتب Z في الجدول Y1، فيجب أن تكون نفس المعاملة T2 هي آخر من يكتب Z في الجدول Y2.

السبب: لأن القيمة النهائية المخزنة في قاعدة البيانات يجب أن تكون نفسها.

3. Read After Write القراءة بعد الكتابة (Update Read):

إذا قرأت معاملة قيمة كتبتّها معاملة أخرى، فيجب الحفاظ على نفس علاقة القراءة في الجدول المكافئ.

مثال:

إذا كان لدينا:

▪ T2 تقرأ Z و T1 تكتب على Z

فهذا يعني:

▪ T2 قرأت القيمة التي كتبتها T1

في أي جدول مكافئ ظاهرياً:

▪ يجب أن تبقى T1 قبل T2

▪ لا يجوز تغيير هذه العلاقة

الهدف: الحفاظ على من قرأ من من (Read-From Relationship).

مفاهيم أخرى من التسلسلية

(Other Notions of Serializability)

▪ لدينا مخطط تنفيذ (Schedule) غير تسلسلي.

T_1	T_5
read(A) $A := A - 50$ write(A)	read(B) $B := B - 10$ write(B)
read(B) $B := B + 50$ write(B)	read(A) $A := A + 10$ write(A)

▪ هذا المخطط يعطي نفس النتيجة النهائية التي يعطيها المخطط التسلسلي $\langle T_1, T_5 \rangle$.

▪ لكن رغم ذلك، هذا المخطط ليس مكافئاً بالتصادم (Conflict Equivalent) ولا مكافئاً ظاهرياً (View Equivalent) للمخطط التسلسلي.

بعض الطرق لم تغلق لتمنعك، بل لتوجهك

- كل من View Serializability و Conflict Serializability يعتمد فقط على تحليل عمليات القراءة والكتابة (Read/Write).
- في بعض الحالات، قد تحتوي المعاملات على عمليات أخرى (مثل الحسابات أو التحديثات الداخلية) لا تؤخذ بعين الاعتبار في هذين النوعين من التحليل.
- لذلك، قد يكون مخطط التنفيذ صحيحاً من حيث النتيجة النهائية (يعطي نفس مخرجات مخطط التنفيذ التسلسلي)، لكنه لا يُعتبر مكافئاً بالتصادم أو ظاهرياً.
- يتطلب تحديد تكافؤ تنفيذ المخطط لتنفيذ مخطط متعاقب تحليل العمليات الأخرى غير الكتابة والقراءة وهو بشكل عام صعب التحقيق وباهظ الثمن.

اختبار إمكانية تسلسل التنفيذ testing for serializability

هذه الفقرة تشرح طريقة اختبار قابلية التسلسل (Testing for Serializability) باستخدام أداة تسمى بيان السوابق (Precedence Graph):

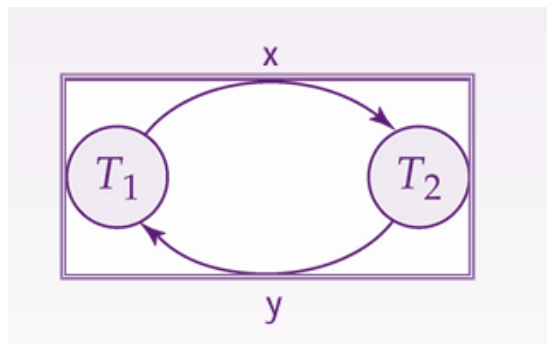
- لدينا مجموعة من المعاملات $T_1, T_2, \dots, T_n$
- نريد أن نعرف هل المخطط (Schedule) الناتج عن تنفيذ هذه المعاملات بشكل متداخل هو قابل للتسلسل (Serializable) أم لا.

بيان السوابق (Precedence Graph):

هو رسم بياني موجه (Directed Graph):

- العُقد (Vertices):** تمثل المعاملات نفسها. ($T_1, T_2, \dots$)
- الأقواس (Arcs):** نرسم قوساً من T_i إلى T_j إذا كان هناك تعارض (Conflict) بينهما، وكان T_i هو الذي نفذ العملية على العنصر أولاً، ويتم ذلك في حال تحقيق أحد الشروط التالية:
 - تنفذ المناقلة T_i تعليمة $write(Q)$ قبل أن تنفذ T_j تعليمة $read(Q)$
 - تنفذ المناقلة T_i تعليمة $read(Q)$ قبل أن تنفذ T_j تعليمة $write(Q)$
 - تنفذ المناقلة T_i تعليمة $write(Q)$ قبل أن تنفذ T_j تعليمة $write(Q)$
- في حال وجد قوس من T_i إلى T_j في بيان السوابق، يعني ذلك أنه في جميع المخططات المتعاقبة S المكافئة ل S يجب أن تنفذ T_i قبل T_j .
- يمكن أن نضع على القوس اسم العنصر Q أو x أو y الذي حدث عليه التعارض

مثال:



مثال لمخطط تنفيذ Schedule A والبيان الموافق له:

T_1	T_2	T_3	T_4	T_5	
read(Y) read(Z)	read(X)			read(V) read(W) read(W)	
	read(Y) write(Y)	write(Z)			
read(U)			read(Y) write(Y) read(Z) write(Z)		
read(U) write(U)					

خطوات تحويل المخطط إلى بيان:

1. ارسم العقد:

من المخطط لدينا المعاملات:

T_1, T_2, T_3, T_4, T_5

إذاً نرسم 5 عقد.

ملاحظة: أحياناً تظهر معاملات لا تدخل في أي تعارض مثل T_5 هنا.

2. حدّد عناصر البيانات المشتركة:

نبحث عن عناصر البيانات التي استخدمت من أكثر من معاملة:

Y
Z

(أما X و V و W فلا تعارض عليها)

3. استخرج التعارضات عنصراً عنصراً:

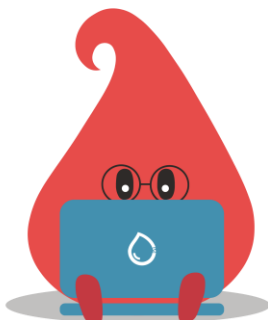
1. عنصر البيانات Y

العمليات على Y:

T1: read(Y)

T2: read(Y), write(Y)

T4: read(Y), write(Y)



التعارضات:

1. $T2$ من $write(Y)$ قبل $T1$ من $read(Y)$

← سهم $T1 \rightarrow T2$

2. $T4$ من $write(Y)$ قبل $T2$ من $write(Y)$

← سهم $T2 \rightarrow T4$

3. $T4$ من $write(Y)$ قبل $T1$ من $read(Y)$

← سهم $T1 \rightarrow T4$

2. عنصر البيانات Z

العمليات على Z:

▪ $T1: read(Z)$

▪ $T3: write(Z)$

▪ $T4: read(Z), write(Z)$

التعارضات:

1. $T3$ من $write(Z)$ قبل $T1$ من $read(Z)$

← سهم $T1 \rightarrow T3$

2. $T4$ من $read(Z)$ قبل $T3$ من $write(Z)$

← سهم $T3 \rightarrow T4$

3. عنصر البيانات U

العمليات على U:

▪ $T1: read(U), write(U)$

▪ لا يوجد أي كتابة من معاملات أخرى

لا تعارض مع معاملات أخرى.

لا نضيف أي أسهم جديدة.

4. تجاهل المعاملة التي لا تدخل في تعارض:

▪ $T5$ تقرأ V و W فقط

▪ لا كتابة مشتركة مع أحد

لا أسهم متصلة بها

5. ارسم الأسهم النهائية كما يظهر في الرسم الموجود في الصورة.

اختبار كون مخطط التنفيذ متسلسلاً تصادميةً:

- مخطط التنفيذ يكون قابلاً للتسلسل تصادميةً إذا وفقط إذا كان مخطط الأولوية الخاص به لا يحتوي على حلقة (Acyclic graph).
- تحتاج خوارزمية كشف الحلقات في بيان مؤلف من n الى n^2 عملية.

مثال:

في Schedule A في الأعلى، أحد الترتيبات الممكنة هو:

$T5 \rightarrow T1 \rightarrow T3 \rightarrow T2 \rightarrow T4$

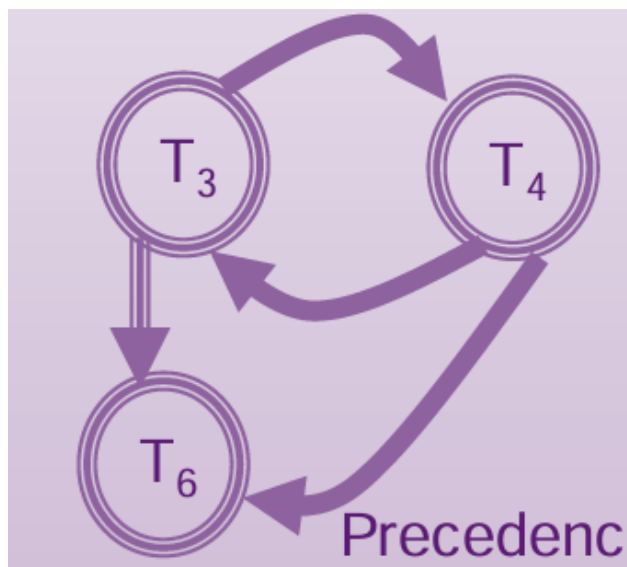
هذا ترتيب تسلسلي صحيح لأنه يتوافق مع ال precedence graph.

اختبار كون مخطط تنفيذ متسلسلاً ظاهرياً:

- لدينا مخطط التنفيذ لمعاملات $T3, T4, T6$:

T_3	T_4	T_6
read(Q)	write(Q)	write(Q)
write(Q)		

- عند بناء بيان السوابق (Precedence Graph)، يظهر أن هناك أقواس (علاقات) بين هذه المعاملات بسبب التعارضات على العنصر Q.



النتيجة: الجدول ليس قابلاً للتسلسل بالتصادم (Not Conflict Serializable)، لكنه قابل للتسلسل بالرؤية (View Serializable).

لماذا الجدول غير قابل للتسلسل بالتصادم؟

الجواب السريع: لأنه ظهرت حلقة في البيان بين T3 و T4.

والحلقة ظهرت لأن:

■ هناك تعارضات بين التعليمات:

• بين T3 (read(Q)) و T4 (write(Q))

• بين T4 (write(Q)) و T6 (write(Q))

■ هذه التعارضات تمنع إعادة ترتيب التعليمات للوصول إلى جدول تسلسلي مكافئ، وبالتالي الجدول ليس Conflict Serializable.

لماذا الجدول قابل للتسلسل بالرؤية؟

■ بالرغم من وجود التعارضات، شروط التكافؤ ظاهرياً (View Equivalence) ما زالت متحققة:

1. أول عملية قراءة كانت في T3، وتبقى كما هي.

2. آخر عملية كتابة كانت في T6، وتبقى كما هي.

3. لا توجد قراءة بعد الكتابة تكسر الشرط.

■ لذلك، الجدول يمكن اعتباره مكافئاً بالرؤية لجدول تسلسلي مثل:

$T3 \rightarrow T4 \rightarrow T6$

ملاحظة: عند وجود كتابات غير مستخدمة أو غير مقروءة يمكن حذفها دون أن تؤثر على النتيجة النهائية.

مثال: إذا كانت الكتابة في T4 لا تُقرأ من أي معاملة لاحقة، فهي كتابة غير مستخدمة، وبالتالي يمكن تجاهلها عند تحليل المخطط أي يمكننا حذف القوس من T4 إلى T3.

بيان السوابق المعدل

دعم بيان السوابق (Modified Precedence Graph):

من أجل اختبار كون مخطط التنفيذ متسلسلاً ظاهرياً (View Serializable)، فإن بيان السوابق العادي لا يكون كافياً، لذلك نقوم بتطوير بيان السوابق ليأخذ بعين الاعتبار شروط التكافؤ الظاهري، ويُسمى هذا البيان ببيان السوابق المعدل.

خطوات تطوير بيان السوابق:

1. إضافة مناقلتين وهما:

نضيف بشكل وهمي مناقلتين إلى مخطط التنفيذ، هما:

■ مناقلة البداية (Tb): تحتوي تعليمات كتابة write(Q) لكل عنصر بيانات Q تم استخدامه في المخطط، وتمثل القيم الابتدائية في قاعدة البيانات.

- **مناقلة النهاية (Tf):** تحتوي تعليمات قراءة (read(Q) لكل عنصر بيانات Q تم استخدامه في المخطط، وتمثل القيم النهائية بعد انتهاء التنفيذ.

الهدف من Tb و Tf هو تمثيل مفهومي:

■ Initial Read

■ Final Write

بشكل صريح في البيان.

2. إضافة الأقواس الإجبارية (Label = 0):

نضيف قوساً من المناقلة T_i إلى المناقلة T_j مع لاحقة 0 بالشكل:

$T_i \longrightarrow 0 \longrightarrow T_j$

وذلك في حال:

- قامت المناقلة T_j بقراءة قيمة عنصر البيانات Q

- وكانت هذه القيمة قد كُتبت سابقاً في المناقلة T_i

اللاحقة (0) تعني أن هذا القوس إجباري ولا يمكن حذفه، لأنه يمثل علاقة قراءة بعد كتابة (Read-From) يجب الحفاظ عليها لتحقيق التكافؤ الظاهري.

3. حذف المناقلات غير المستخدمة:

نقوم بحذف جميع الأقواس المرتبطة بالمناقلات غير المستخدمة.

نقول أن المناقلة T_i غير مستخدمة إذا:

- لم يوجد أي قوس يربطها بالمناقلة النهائية T_f في بيان السوابق المعدل

أي أن هذه المناقلة لا تؤثر على النتيجة النهائية، وبالتالي يمكن تجاهلها دون التأثير على التكافؤ الظاهري.

4. معالجة الكتابات اللاحقة (الحالة الأساسية):

من أجل كل عنصر بيانات Q بحيث:

- تقوم المناقلة T_j بقراءة قيمة Q المكتوبة سابقاً في المناقلة T_i

- تقوم المناقلة T_k بتنفيذ عملية write(Q)، بحيث $T_k \neq T_b$

أي مثل هذه الحالة التي فيها:

- $T_i = T_b$ (كتب القيمة الابتدائية)

- $T_j = T_3$ (قرأ القيمة التي كتبها T_b)

- $T_k = T_4$ كتب نفس Q

فهنا لا يجوز أن تكون T_4 بين T_b و T_3 .

T_3	T_4
read(Q)	write(Q)
write(Q)	

ومنه توجد حالة قد تنكسر الرؤية إذا جاءت كتابة Tk بين Ti و Tj، لذلك يجب فرض قيود ترتيب مناسبة وحينها ننفذ ما يلي:

↪ الحالات الممكنة:

الحالة (1):

إذا كان: $T_f \neq T_j$ و $T_b = T_i$

نقوم بإضافة القوس في بيان السوابق المعدل: $T_j \longrightarrow 0 \longrightarrow T_k$

الحالة (2):

إذا كان: $T_f = T_j$ و $T_b \neq T_i$

نقوم بإضافة القوس في بيان السوابق المعدل: $T_k \longrightarrow 0 \longrightarrow T_i$

الحالة (3) – الحالة العامة:

إذا كان: $T_f \neq T_j$ و $T_b \neq T_i$

نقوم بإضافة القوسين في بيان السوابق المعدل: $T_j \longrightarrow \rho \longrightarrow T_k$ و $T_k \longrightarrow \rho \longrightarrow T_i$

حيث ρ عدد صحيح موجب ($\rho > 0$)، ويدل على أن هذه الأقواس غير إجبارية.

التفسير المنطقي للحالة (3):

- إذا قامت المناقلة T_i بكتابة بيانات، وقامت المناقلة T_j بقراءتها، ثم قامت المناقلة T_k بكتابة نفس البيانات، فإنه يجب تنفيذ Tk إما قبل T_i أو بعد T_j ، ولا يجوز أن تأتي Tk بينهما لأنها ستكسر الرؤية.
- تطبيق القاعدة (3) لا يعني أن Tk تأتي قبل T_i وبعد T_j معاً، وإنما يعني أن لدينا حرية اختيار إحدى الحالتين فقط بما يمنع كسر الرؤية.

ملاحظة هامة:

الحالتان (1) و (2) هما حالات خاصة ناتجة عن كون:

T_b هي المناقلة الأولى في التسلسل.

T_f هي المناقلة الأخيرة في التسلسل.

توضيح إضافي للخطوة 4 لتقريب الفكرة أكثر:

عندنا 3 مناقلات حول نفس عنصر البيانات Q:

1. T_i : كتبت Q.
2. T_j : قرأت Q (وقرأت تحديداً القيمة التي كتبتها T_i)
3. T_k : كتبت Q أيضاً (بعدهم أو بينهم).

المشكلة:

- إذا جاءت كتابة Tk بين كتابة T_i وقراءة T_j
- T_j لن ترى قيمة T_i ← تنكسر الرؤية (view).

إذا:

كتابة Tk ممنوعة أن تكون في الوسط.

ومنه نستنتج القاعدة التي تقول أنه إذا:

- T_i كتب
 - T_j قرأ
 - T_k كتب نفس العنصر
- فإن T_k يجب أن تكون:
- إما قبل T_i
 - أو بعد T_j
- ولا يجوز أن تكون بينهما.

مثال:

لدينا مخطط التنفيذ التالي:

T_3	T_4
read(Q)	write(Q)
write(Q)	

الخطوة الأولى حتى نرسم البيان السابق ونختبر أنه متسلسل ظاهرياً:

- إضافة المناقلتين الوهميتين T_b و T_f .
- نفعل هذا دائماً قبل أي رسم.

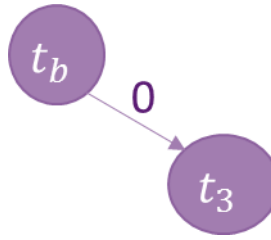
T_b مناقلة البداية:

- نعتبرها كأنها كتبت كل عناصر البيانات في البداية
- تمثل Initial Read
- أول قراءة لا تجد كتابة قبلها ← نعتبر أنها قرأت من T_b وهذا يعني القيمة جاءت من قاعدة البيانات قبل بدء التنفيذ

T_f مناقلة النهاية:

- نعتبرها كأنها تقرأ كل عناصر البيانات في النهاية
- تمثل Final Write
- آخر كتابة لا يوجد بعدها قراءة ← نربطها بـ T_f وهذا يعني هذه القيمة هي التي ستبقى في قاعدة البيانات في النهاية

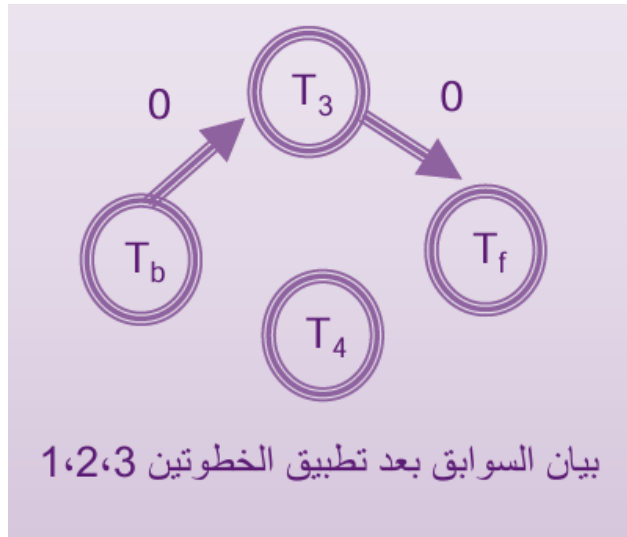
بداية نرسم label من t_b الى t_3 (اول قراءة لا يوجد قبلها كتابة) ونضع على القوس 0 لأنه قوس اجباري (يمثل عملية قراءة بعد كتابة يعني T_b كتبت و T_3 قرأت):



ومن ثم نرسم label من t_3 الى t_f (آخر عملية كتابة لا يوجد بعدها قراءة) ونضيف 0 على القوس ايضاً:



يصبح لدينا:



بالنسبة ل T_4 :

نلاحظ أن الحالة هنا تطابق الحالة 1 من الخطوة 4:

$$T_i = T_b$$

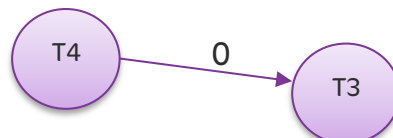
$$T_j = T_3 \quad (T_j \neq T_f)$$

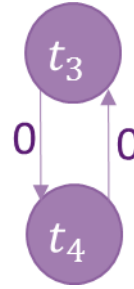
$$T_k = T_4$$

إذاً نقوم بإضافة القوس: $T_j \xrightarrow{0} T_k$

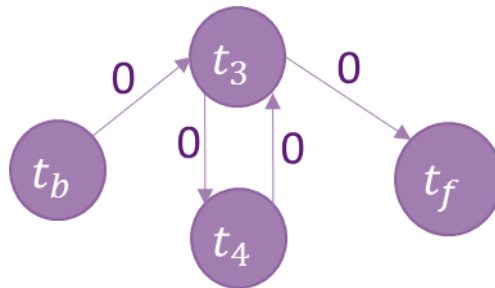


ونلاحظ انه من المخطط يوجد عملية كتابة في T_4 بعد القراءة في T_3 ← حالة تعارض (WRITE AFTER READ):
ومنه حسب قاعدة بيان السوابق العادي (الخاص بالتسلسل التصادمي):





ويكون المخطط النهائي:



بما أن لدينا حلقة فالمخطط غير تسلسلي ظاهرياً.

النتيجة: بيان يحوي حلقة وهو غير متسلسل ظاهرياً

إذا وجد بيان لمخطط تنفيذ لا يحوي حلقة ← يكون مخطط التنفيذ متسلسل ظاهرياً.

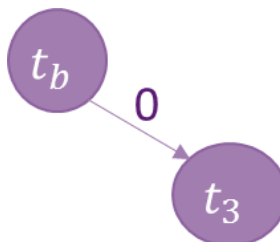
مثال 2:

ليكن لدينا المخطط التنفيذي التالي، اختبر هل المخطط متسلسل ظاهرياً:

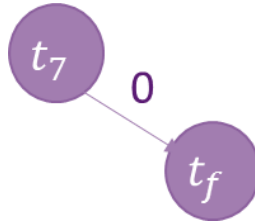
T_3	T_4	T_7
read(Q)	write(Q)	read(Q)
write(Q)		write(Q)

كالمثال السابق نضيف مناقلتين وهما t_b و t_f ونبدأ برسم بيان السوابق:

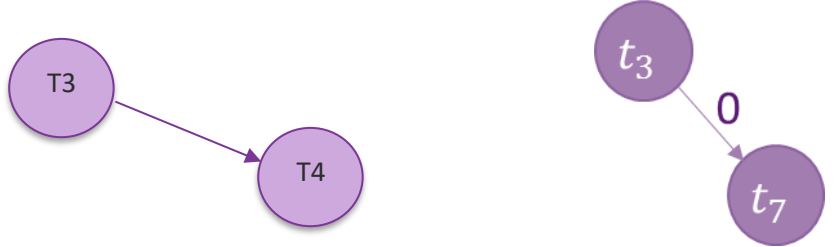
نرسم label من t_b الى t_3 (اول عملية قراءة لم تسبقها كتابة T_b هي التي كتبت)



ونرسم label من t_7 الى t_f (اخر عملية كتابة لم يأتي بعدها قراءة):



ولدينا التعارضات بين T_7 و T_3 و بين T_4 و T_3 :



ولدينا التعارضات بين T_7 و T_4 (write after write) و بين T_4 و T_3 (write after write):



والتعارض بين T_7 و T_3 (write after read):



وبتطبيق قاعدة بيان السوابق المعدل:

عندما تكون لدينا الحالة التالية:

■ قراءة بعد كتابة

■ $T_i \neq T_b$

تظهر حالتنا ممكنتان:

1. T_4 قبل T_3

2. T_7 بعد T_3

أي أن العلاقة غير مؤكدة.

ماذا نفعل عندما تكون العلاقة غير مؤكدة؟

نرسم القوس، لكن نضع عليه 1 وليس 0، لأن:

■ 0 = قوس إجباري (must)

■ 1 = قوس اختياري / محتمل (may)

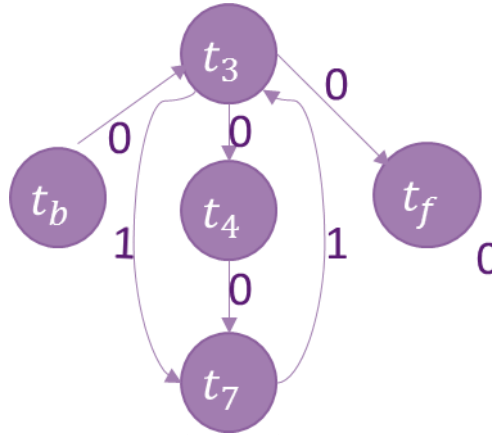
■ $T_i \neq T_f$

■ والكتابة يمكن تجاوزها (Overwritten)

النتيجة النهائية:

- الأسهم من T_b و T_f = 0
- التعارضات المؤكدة = 0
- العلاقات الناتجة عن كتابة يمكن تجاوزها = 1

ولهذا يظهر في البيان النهائي قوس بقيمة 1، لأنه لا يوجد ترتيب واحد مضمون يحافظ على الرؤية ويكون البيان الناتج:



وهنا، وقبل أن نحكم فيما إذا كان المخطط متسلسلاً ظاهرياً أم لا، يجب الانتباه إلى أن:

- عند تطبيق القاعدة (3-4) عدد n من المرات قد نحصل على عدة بيانات سوابق معدلة (حتى 2^n) وذلك لأن كل قوس ذي قيمة (1) يمثل ترتيباً اختيارياً.
- وبالتالي لا يكفي وجود حلقة واحدة للحكم بعدم التسلسلية الظاهرية.
- بل يكفي وجود بيان سوابق واحد فقط خالٍ من الحلقات لنقول إن مخطط التنفيذ متسلسل ظاهرياً

يعني:

رسم بيان فيه:

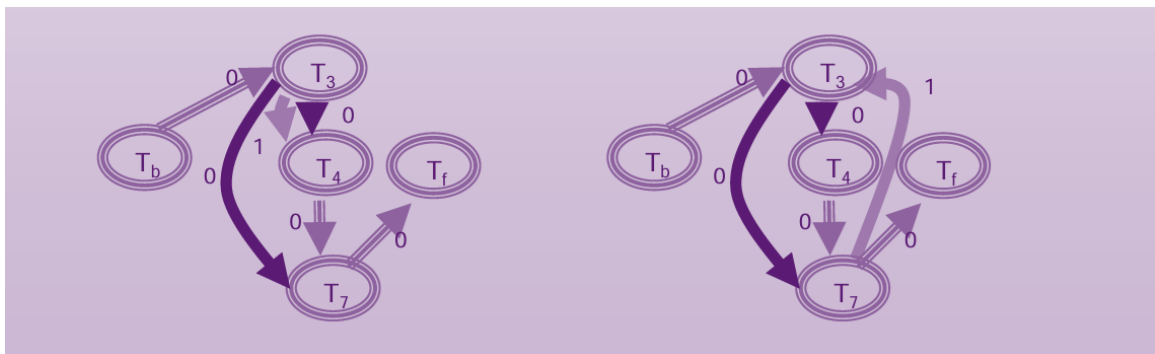
- أقواس 0 فقط ← وجود أي حلقة ← فشل مباشر
- أقواس 1 موجودة ← لازم نجرب كسر الحلقة

وفي مثالنا يمكن كسر الحلقة عبر:

اختيار أحد الترتيبين الممكنين للقوس ذي القيمة (1)، أي:

- إما تنفيذ T_4 قبل T_3
- أو تنفيذ T_4 بعد T_7

وبما أن هذا القوس غير إجباري (قيمه 1)، فنحن أحرار في اختيار أحد الترتيبين.



فإذا اخترنا الترتيب الذي:

- لا يتعارض مع الأقواس ذات القيمة (0)
 - ولا يُولد دورة (Cycle)
- نحصل على بيان سوابق بلا حلقة.

وبالتالي:

النتيجة النهائية: بما أنه يوجد على الأقل بيان سوابق معدّل واحد بدون حلقة، فإن مخطط التنفيذ في مثالنا متسلسل ظاهرياً

فائدة أربيسيزية:

يساعد الفرز الطوبولوجي (Topological Sorting) في استخراج ترتيب أو أكثر لتنفيذ المناقلات انطلاقاً من بيان السوابق المعدّل، بحيث يحترم جميع القيود المفروضة بين المناقلات ولا يحتوي على أي حلقة.

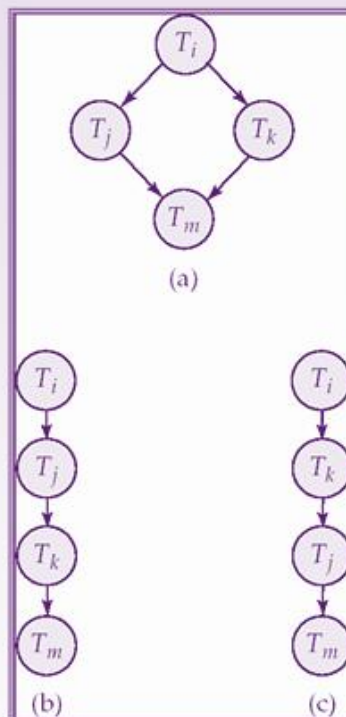
في حال احتوى البيان على أقواس إجبارية (0) فقط، فإن الفرز الطوبولوجي يعطي ترتيباً جيداً أو عدداً محدوداً جداً من الترتيبات.

أما عند وجود أقواس غير إجبارية (1) ، كما في مثالنا، فإن ذلك يفتح المجال أمام أكثر من ترتيب صحيح للمناقلات، لأن هذه الأقواس تمثل اختيارات ترتيب بديلة لا تكسر الرؤية الظاهرية.

وبالتالي، إذا أمكن عبر أحد هذه الترتيبات الناتجة عن الفرز الطوبولوجي الحصول على بيان بلا حلقة، فإن مخطط التنفيذ يُعد متسلسلاً ظاهرياً، حتى لو ظهرت حلقات في ترتيبات أخرى.

وهنا تتجلى أهمية بيان السوابق المعدّل والفرز الطوبولوجي معاً في كشف المرونة الحقيقية في ترتيب المناقلات

Illustration of Topological Sorting



قابلية الاسترجاع والاستعادة recoverability

- عند تنفيذ المعاملات بشكل متزامن، قد تعتمد معاملة على بيانات كتبتها معاملة أخرى.
- إذا فشلت المعاملة الأولى وتم عمل Rollback لها، فإن المعاملة الثانية التي قرأت منها تصبح غير صحيحة أيضاً، وقد تحتاج هي الأخرى إلى التراجع.
- لذلك يجب ضمان أن المخطط قابل للاسترجاع (Recoverable Schedule).

الجدول القابلة للاسترجاع (Recoverable Schedules):

- الجدول يكون قابلاً للاسترجاع إذا تحقق الشرط:
 - إذا قرأت معاملة T_j بيانات كتبتها معاملة T_i ، فيجب أن تنفذ عملية Commit في T_i قبل أن تنفذ عملية Commit في T_j .
- هذا يضمن أن أي بيانات تمت قراءتها من T_i هي بيانات مؤكدة وليست معرضة للتراجع.

مثال توضيحي (حالة غير قابلة للاستعادة):

T_8	T_9
read(A) write(A) read(B)	read(A)

- تقوم المناقلة T_9 بقراءة قيمة من T_8
- ثم تقوم T_9 بتنفيذ COMMIT مباشرة بعد القراءة
- لاحقاً يحدث عطل أثناء تنفيذ T_8 ويتم تنفيذ ROLLBACK لها

في هذه الحالة:

- تكون T_9 قد استخدمت قيمة غير مؤكدة
- وتصبح قاعدة البيانات في حالة غير متسقة (Inconsistent)
- لذلك يكون هذا المخطط غير قابل للاستعادة.

استنتاج هام:

يجب التأكد أن جميع مخططات التنفيذ التي تُنفَّذ على قاعدة البيانات هي مخططات قابلة للاستعادة (Recoverable Schedules)

⚡ مشكلة التراجع المتسلسل/العودة بشكل شلال (Cascading Rollback):

إذا فشلت معاملة واحدة، قد يؤدي ذلك إلى سلسلة من التراجعات (Rollbacks) في معاملات أخرى اعتمدت عليها.

مثال:

T_{10}	T_{11}	T_{12}
read(A) read(B) write(A)	read(A) write(A)	read(A)

إذا فشلت المعاملة T_{10} :

■ يجب أن تتراجع أيضًا المعاملة T_{11} لأنها قرأت من T_{10} .

■ ويجب أن تتراجع المعاملة T_{12} لأنها اعتمدت على نتائج T_{11} .

النتيجة: سلسلة من التراجعات تؤدي إلى فقدان كبير في العمل المنجز.

الحل: الجداول الخالية من التراجع المتسلسل (Cascadeless Schedules) / مخططات تنفيذ لمناقلات غير مترابطة بشكل شلال:

■ الجدول يكون Cascadeless إذا تحقق الشرط:

• لا يُسمح لأي معاملة بقراءة بيانات من معاملة أخرى إلا بعد أن تنفذ تلك المعاملة عملية Commit.

■ هذا يمنع حدوث التراجع المتسلسل/بشكل شلال، ويجعل النظام أكثر استقرارًا.

■ كل جدول Cascadeless هو أيضًا Recoverable، لكنه أكثر أمانًا لأنه يمنع سلسلة التراجعات.

تنفيذ العزلة Isolation

يوجد طريقتين لتحقيق العزلة في تنفيذ المناقلات:

■ إقفال قاعدة المعطيات بشكل كامل عند تنفيذ مناقلة وهذا غير فعال لأنه يجب أن نسعى لزيادة الأداء مع

المحافظة على قاعدة المعطيات متلائمة وهذا الحل غير فعال.

■ يجب أن يكون مخطط التنفيذ مخططًا متسلسلاً تصادميةً أو متسلسلاً ظاهرياً وقابل للاستعادة ومن الأفضل أن

يكون غير مترابط بشكل شلال من أجل الحفاظ على قاعدة المعطيات متلائمة وهذه هي الطريقة الأفضل وهو

حل فعال.

الوقت لا يصنع، نحن من نصنعه ثم نبحث عنه

تعريف المناقلات في لغة SQL

- تحوي لغة التعامل مع المعطيات DML تعابير لتحديد مجموعة التعليمات المكونة لمناقلة
- تبدأ المناقلة بلغة SQL بشكل ضمني (ليست بحاجة لتصريح)
- تنتهي المناقلة بلغة SQL ب :
 - Commit work لتسجيل المناقلة الحالية (تسجيل التعديلات على قاعدة المعطيات) والبدء في مناقلة جديدة.
 - Rollback work لإنهاء عمل المناقلة الحالية والعودة إلى الحالة السابقة لقاعدة المعطيات

The End...

في اللحظة التي تقرر فيها أن تتحمل مسؤولية حياتك، تبدأ فعلياً أول خطوة نحو مستقبلك الحقيقي

